

```
mysql> use cg_mysql;
Database changed
mysql> /* Set Operators
/*> 1. Union
/*> 2. Union all
/*> 3. Intersect
/*> 4. Minus/Except
/*> */
mysql> -- Union: Combines results from two queries and removes duplicate rows.
Columns must have the same number and type.
mysql> -- Union all: Combines results from two queries and include all rows.
including duplicates. columns must have the same number and type.
mysql> -- Intersect: Returns rows that are common to both queries. columns must
have the same number and type. SQL doesn't support intersect directly.
mysql> -- Minus/Except: Returns rows from the first query that are not present in
the second query. Columns must have the same number and type.
mysql> drop table students;
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> CREATE TABLE Students (
->     student_id INT PRIMARY KEY,
->     student_name VARCHAR(50),
->     course VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql>
mysql> INSERT INTO Students VALUES
-> (101,'Rahul','Java Full Stack'),
-> (102,'Priya','Python Full Stack'),
-> (103,'Kiran','Data Science'),
-> (104,'Sneha','Java Full Stack'),
-> (105,'Ajay','Testing');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql>
mysql> CREATE TABLE Placed_Students (
->     student_id INT PRIMARY KEY,
->     student_name VARCHAR(50),
->     company_name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Placed_Students VALUES
-> (103,'Kiran','TCS'),
-> (104,'Sneha','Infosys'),
-> (105,'Ajay','Wipro'),
-> (106,'Pooja','Accenture'),
```

```
-> (107, 'Ramesh', 'Capgemini');
Query OK, 5 rows affected (0.05 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> select * from students;
```

| student_id | student_name | course            |
|------------|--------------|-------------------|
| 101        | Rahul        | Java Full Stack   |
| 102        | Priya        | Python Full Stack |
| 103        | Kiran        | Data Science      |
| 104        | Sneha        | Java Full Stack   |
| 105        | Ajay         | Testing           |

```
5 rows in set (0.04 sec)
```

```
mysql> select * from placed_students;
```

| student_id | student_name | company_name |
|------------|--------------|--------------|
| 103        | Kiran        | TCS          |
| 104        | Sneha        | Infosys      |
| 105        | Ajay         | Wipro        |
| 106        | Pooja        | Accenture    |
| 107        | Ramesh       | Capgemini    |

```
5 rows in set (0.00 sec)
```

```
mysql> -- Union
```

```
mysql> select student_name from students union select student_name from placed_students;
```

| student_name |
|--------------|
| Rahul        |
| Priya        |
| Kiran        |
| Sneha        |
| Ajay         |
| Pooja        |
| Ramesh       |

```
7 rows in set (0.06 sec)
```

```
mysql> -- Union all
```

```
mysql> select student_name from students union all select student_name from placed_students;
```

| student_name |
|--------------|
| Rahul        |
| Priya        |
| Kiran        |
| Sneha        |
| Ajay         |
| Pooja        |
| Ramesh       |
| Kiran        |
| Sneha        |
| Ajay         |
| Pooja        |
| Ramesh       |

|        |
|--------|
| Rahul  |
| Priya  |
| Kiran  |
| Sneha  |
| Ajay   |
| Kiran  |
| Sneha  |
| Ajay   |
| Pooja  |
| Ramesh |

+-----+

10 rows in set (0.04 sec)

mysql> -- Intersect

mysql> select student\_name from students intersect select student\_name from placed\_students;

+-----+

|              |
|--------------|
| student_name |
| Kiran        |
| Sneha        |
| Ajay         |

+-----+

3 rows in set (0.00 sec)

mysql> -- Except

mysql> select student\_name from students except select student\_name from placed\_students;

+-----+

|              |
|--------------|
| student_name |
| Rahul        |
| Priya        |

+-----+

2 rows in set (0.00 sec)

mysql> /\* Joins

/\*> A join in MySQL is used to combine rows from two or more tables based on related columns.

/\*>

/\*> example

/\*> -----

/\*> employees table--->Employee details.

/\*> departments table---> department details.

/\*>

/\*> if you want to see which employee belongings to which department. you need to join these two tables.

/\*>

/\*> Why do we need joins?

/\*> -----

```

/*> 1. to fetch data from multiple tables in a single Query.
/*> 2. to reduce data duplication(instead of keeping everything in one table.)
/*> 3. to implement relationships like(one-to-one, one-to-many, many-to-one,
many-to-many) in databases.
/*> 4. to generate reports and analytics combining different datasets.
/*>
/*> types of joins in MySQL
/*> -----
/*> 1.inner join
/*> 2.left join
/*> 3. right join
/*> 4. full join
/*> 5. cross join
/*> 6. self join
/*> */
mysql> -- Inner Join: Returns only the rows that have matching values in both
tables.
mysql> -- Left Outer Join: Returns all rows from the left table and the matched
rows from the right table. If no match, NULL values appear for right table columns.
mysql> -- Right Outer Join: Returns all rows from the right table and the matched
rows from the left table. If no match, NULL values appear for left table columns.
mysql> -- Full Outer Join: To simulate, use the UNION of LEFT JOIN and RIGHT JOIN.
mysql> -- Cross Join: Returns Cartesian product ? every row from the first table
with every row from the second table.
mysql> -- Self Join: A table joined with itself.
mysql> create table tab_1(
  -> num_id int
  -> );
Query OK, 0 rows affected (0.13 sec)

mysql> create table tab_2(
  -> num_id int
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> insert into tab_1 values(10,11,12,14);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> insert into tab_1 values(10),(11),(12),(14);
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> insert into tab_2 values(11),(12),(13),(15);
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> select * from tab_1;
+-----+
| num_id |
+-----+
|      10 |

```

|    |
|----|
| 11 |
| 12 |
| 14 |

```
+-----+
```

4 rows in set (0.00 sec)

```
mysql> select * from tab_2;
```

|        |
|--------|
| num_id |
|--------|

```
+-----+
```

|    |
|----|
| 11 |
| 12 |
| 13 |
| 15 |

```
+-----+
```

4 rows in set (0.00 sec)

```
mysql> -- inner join
```

```
mysql> select * from tab_1 inner join tab_2 on tab_1.num_id = tab_2.num_id;
```

```
+-----+-----+
```

|        |        |
|--------|--------|
| num_id | num_id |
|--------|--------|

```
+-----+-----+
```

|    |    |
|----|----|
| 11 | 11 |
| 12 | 12 |

```
+-----+-----+
```

2 rows in set (0.05 sec)

```
mysql> select * from tab_1 left join tab_2 on tab_1.num_id = tab_2.num_id;
```

```
+-----+-----+
```

|        |        |
|--------|--------|
| num_id | num_id |
|--------|--------|

```
+-----+-----+
```

|    |      |
|----|------|
| 10 | NULL |
| 11 | 11   |
| 12 | 12   |
| 14 | NULL |

```
+-----+-----+
```

4 rows in set (0.03 sec)

```
mysql> select * from tab_1 right join tab_2 on tab_1.num_id = tab_2.num_id;
```

```
+-----+-----+
```

|        |        |
|--------|--------|
| num_id | num_id |
|--------|--------|

```
+-----+-----+
```

|      |    |
|------|----|
| 11   | 11 |
| 12   | 12 |
| NULL | 13 |
| NULL | 15 |

```
+-----+-----+
```

4 rows in set (0.00 sec)

```
mysql> select * from tab_1 left join tab_2 on tab_1.num_id = tab_2.num_id
```

-> union

-> select \* from tab\_1 right join tab\_2 on tab\_1.num\_id = tab\_2.num\_id;

| num_id | num_id |
|--------|--------|
| 10     | NULL   |
| 11     | 11     |
| 12     | 12     |
| 14     | NULL   |
| NULL   | 13     |
| NULL   | 15     |

6 rows in set (0.00 sec)

mysql> select \* from tab\_1 cross join tab\_2;

| num_id | num_id |
|--------|--------|
| 14     | 11     |
| 12     | 11     |
| 11     | 11     |
| 10     | 11     |
| 14     | 12     |
| 12     | 12     |
| 11     | 12     |
| 10     | 12     |
| 14     | 13     |
| 12     | 13     |
| 11     | 13     |
| 10     | 13     |
| 14     | 15     |
| 12     | 15     |
| 11     | 15     |
| 10     | 15     |

16 rows in set (0.00 sec)

mysql> notee